Java Streams Interview Problems

60 problems with solutions and explanations

30 for Java 8 | 30 for Java 17

Each problem has a statement, an example, the solution code and a short explanation. Part 1 uses only Java 8 APIs. Part 2 uses newer features: Stream.toList(), takeWhile, Collectors.teeing, records, pattern-matching instanceof, Map.entry, String.repeat.

Part 1: Java 8 (30 problems)

Compatible with Java 8: no var, List.of, Stream.toList(), takeWhile, teeing or String.repeat.

Shared setup code

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;
import static java.util.stream.Collectors.*;

// Java 8 compatible: no var, List.of, Stream.toList(), takeWhile, teeing, String.repeat
public class Java8Streams {

    static class Emp {
        final String name, dept; final int salary, age;
        Emp(String name, String dept, int salary, int age) {
            this.name = name; this.dept = dept; this.salary = salary; this.age = age;
        }
        String getName() { return name; }
        String getDept() { return dept; }
        int getSalary() { return salary; }
        int getAge() { return age; }
        @Override public String toString() { return name; }
    }

    static final List<Emp> EMPS = Arrays.asList(
        new Emp("A", "Eng", 100, 30), new Emp("B", "Eng", 150, 40),
        new Emp("C", "Eng", 120, 35), new Emp("D", "Eng", 150, 28),
        new Emp("E", "HR", 80, 45),   new Emp("F", "HR", 90, 38),
        new Emp("G", "Ops", 70, 25));
```

1. Sum of squares of even numbers

PROBLEM

Given a list of integers, return the sum of the squares of the even numbers.

EXAMPLE

```
[5,3,8,1,9,2,8]  ->  132
```

SOLUTION

```
static int sumSquaresOfEvens(List<Integer> l) {
    return l.stream().filter(n -> n % 2 == 0).mapToInt(n -> n * n).sum();
}
```

EXPLANATION

filter keeps the evens, mapToInt squares each into a primitive stream, and sum() adds them (64 + 4 + 64). Using the primitive stream avoids boxing.

2. Min, max, average, sum, count in one go

PROBLEM

Return the min, max, average, sum and count of a list of integers in a single pass.

EXAMPLE

```
[5,3,8,1,9,2,8]  ->  min=1, max=9, avg=5.14, sum=36, count=7
```

SOLUTION

```
static IntSummaryStatistics stats(List<Integer> l) {
    return l.stream().mapToInt(Integer::intValue).summaryStatistics();
}
```

EXPLANATION

mapToInt turns the list into an IntStream and summaryStatistics() computes all five values in one traversal, so the list is never iterated more than once.

3. Strings starting with a char, uppercased, sorted

PROBLEM

From a list of strings, keep those starting with a given character, uppercase them and sort them.

EXAMPLE

```
[banana, apple, avocado, cherry], 'a'  ->  [APPLE, AVOCADO]
```

SOLUTION

```
static List<String> upperSorted(List<String> l, char c) {
    return l.stream().filter(s -> !s.isEmpty() && s.charAt(0) == c)
        .map(String::toUpperCase).sorted().collect(toList());
}
```

EXPLANATION

filter on the first character (guarding against empty strings), map to upper case with String::toUpperCase, sorted() uses natural order, and collect(toList()) builds the result.

4. Join employee names as [A, B, C]

PROBLEM

Print all employee names as a single string in the form [A, B, C].

EXAMPLE

```
EMPS  ->  [A, B, C, D, E, F, G]
```

SOLUTION

```
static String joinNames(List<Emp> l) {
    return l.stream().map(Emp::getName).collect(joining(", ", "[", "]"));
}
```

EXPLANATION

map extracts names and Collectors.joining(delimiter, prefix, suffix) concatenates them with the separator and the surrounding brackets.

5. All pairs of values summing to k

PROBLEM

Find all pairs (by position, i < j) in a list whose values add up to k.

EXAMPLE

```
[5,3,8,1,9,2,8], k=10  ->  [(8,2), (1,9), (2,8)]
```

SOLUTION

```
static List<String> pairsWithSum(List<Integer> l, int k) {
    return IntStream.range(0, l.size()).boxed()
        .flatMap(i -> IntStream.range(i + 1, l.size())
            .filter(j -> l.get(i) + l.get(j) == k)
            .mapToObj(j -> "(" + l.get(i) + "," + l.get(j) + ")"))
        .collect(toList());
}
```

EXPLANATION

An outer stream of indices flatMaps into an inner stream of later indices j > i. The filter keeps pairs summing to k and mapToObj formats them. flatMap turns the nested streams into one flat result. Complexity is O(n^2).

6. Character counts (insertion ordered)

PROBLEM

Count how many times each character appears in a string, keeping first-seen order.

EXAMPLE

```
"swiss"  ->  {s=3, w=1, i=1}
```

SOLUTION

```
static Map<Character, Long> charCount(String s) {
    return s.chars().mapToObj(c -> (char) c)
        .collect(groupingBy(Function.identity(), LinkedHashMap::new, counting()));
}
```

EXPLANATION

chars() gives an IntStream, mapToObj casts each to Character, and groupingBy with a LinkedHashMap factory and counting() builds an insertion-ordered frequency map.

7. First non-repeating character

PROBLEM

Return the first non-repeating character of a string.

EXAMPLE

```
"swiss"  ->  w
```

SOLUTION

```
static Optional<Character> firstUnique(String s) {
    return charCount(s).entrySet().stream()
        .filter(e -> e.getValue() == 1).map(Map.Entry::getKey).findFirst();
}
```

EXPLANATION

Reuse the ordered frequency map from problem 6. Because it is a LinkedHashMap, the first entry with count 1 is the first unique character in the original string.

8. Second highest distinct number

PROBLEM

Find the second highest distinct number in a list.

EXAMPLE

```
[5,3,8,1,9,2,8]  ->  8
```

SOLUTION

```
static Optional<Integer> secondHighest(List<Integer> l) {
    return l.stream().distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst();
}
```

EXPLANATION

distinct() removes duplicates (so a repeated maximum does not count twice), sorted(reverseOrder()) puts the largest first, skip(1) drops the maximum, and findFirst() returns an Optional.

9. Group employees by department

PROBLEM

Group employees by department.

EXAMPLE

```
EMPS  ->  {Eng=[A,B,C,D], HR=[E,F], Ops=[G]}
```

SOLUTION

```
static Map<String, List<Emp>> byDept(List<Emp> l) {
    return l.stream().collect(groupingBy(Emp::getDept));
}
```

EXPLANATION

groupingBy(classifier) collects elements into a Map whose keys are the classifier results and whose values are lists of matching elements.

10. Employee count per department

PROBLEM

Count the employees in each department.

EXAMPLE

```
EMPS  ->  {Eng=4, HR=2, Ops=1}
```

SOLUTION

```
static Map<String, Long> countByDept(List<Emp> l) {
    return l.stream().collect(groupingBy(Emp::getDept, counting()));
}
```

EXPLANATION

groupingBy with the downstream collector counting() replaces the default list with a Long count per group.

11. Average salary per department

PROBLEM

Compute the average salary per department.

EXAMPLE

```
EMPS  ->  {Eng=130.0, HR=85.0, Ops=70.0}
```

SOLUTION

```
static Map<String, Double> avgSalary(List<Emp> l) {
    return l.stream().collect(groupingBy(Emp::getDept, averagingInt(Emp::getSalary)));
}
```

EXPLANATION

groupingBy with averagingInt(Emp::getSalary) as downstream collector computes the mean for each department.

12. Highest paid employee per department

PROBLEM

Find the highest paid employee in each department.

EXAMPLE

```
EMPS  ->  {Eng=B, HR=F, Ops=G}
```

SOLUTION

```
static Map<String, Emp> topPaid(List<Emp> l) {
    return l.stream().collect(toMap(Emp::getDept, Function.identity(),
        BinaryOperator.maxBy(Comparator.comparingInt(Emp::getSalary))));
}
```

EXPLANATION

toMap(key, value, mergeFunction) resolves duplicate keys with BinaryOperator.maxBy(comparator), which keeps the employee with the larger salary. On ties it keeps the earlier one, which is why B beats D.

13. Partition names by salary > 100

PROBLEM

Split employee names into two lists: salary above 100, and the rest.

EXAMPLE

```
EMPS  ->  {false=[A,E,F,G], true=[B,C,D]}
```

SOLUTION

```
static Map<Boolean, List<String>> partitionBySalary(List<Emp> l) {
    return l.stream().collect(partitioningBy(e -> e.getSalary() > 100,
        mapping(Emp::getName, toList())));
}
```

EXPLANATION

partitioningBy always returns both a true and false key. The second argument mapping(Emp::getName, toList()) converts employees to names inside each partition.

14. Names sorted by salary desc, then name asc

PROBLEM

List employee names sorted by salary descending, then by name ascending.

EXAMPLE

```
EMPS  ->  [B, D, C, A, F, E, G]
```

SOLUTION

```
static List<String> sortedNames(List<Emp> l) {
    return l.stream()
        .sorted(Comparator.comparingInt(Emp::getSalary).reversed().thenComparing(Emp::getName))
        .map(Emp::getName).collect(toList());
}
```

EXPLANATION

Comparator.comparingInt(getSalary).reversed() sorts by salary high to low and thenComparing(getName) breaks ties alphabetically.

15. Nth highest distinct salary

PROBLEM

Find the nth highest distinct salary.

EXAMPLE

```
n=2  ->  120
```

SOLUTION

```
static Optional<Integer> nthHighestSalary(List<Emp> l, int n) {
    return l.stream().map(Emp::getSalary).distinct()
        .sorted(Comparator.reverseOrder()).skip(n - 1).findFirst();
}
```

EXPLANATION

Map to salaries, distinct() so equal salaries count once, sort descending, skip(n-1) and take the first. Optional is returned because n may exceed the number of distinct salaries.

16. Department with highest total salary

PROBLEM

Find the department with the highest total salary.

EXAMPLE

```
EMPS  ->  Eng (520)
```

SOLUTION

```
static Optional<String> richestDept(List<Emp> l) {
    return l.stream().collect(groupingBy(Emp::getDept, summingInt(Emp::getSalary)))
        .entrySet().stream().max(Map.Entry.comparingByValue()).map(Map.Entry::getKey);
}
```

EXPLANATION

groupingBy with summingInt gives total per department. Then stream the entries, take max(Map.Entry.comparingByValue()) and map to its key.

17. Total salary per dept, sorted desc by total (ordered map)

PROBLEM

Return total salary per department sorted by total descending, preserving that order in the result.

EXAMPLE

```
EMPS  ->  {Eng=520, HR=170, Ops=70}
```

SOLUTION

```
static Map<String, Integer> totalSorted(List<Emp> l) {
    return l.stream().collect(groupingBy(Emp::getDept, summingInt(Emp::getSalary)))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
        .collect(toMap(Map.Entry::getKey, Map.Entry::getValue, (a, b) -> a, LinkedHashMap::new));
}
```

EXPLANATION

A HashMap loses ordering, so sort the entry stream and collect with toMap(..., (a,b)->a, LinkedHashMap::new). The merge function is required by that overload but never fires because keys are already unique.

18. Flatten list of lists

PROBLEM

Flatten a list of lists into one list.

EXAMPLE

```
[[1,2],[3],[4,5]]  ->  [1,2,3,4,5]
```

SOLUTION

```
static <T> List<T> flatten(List<List<T>> l) {
    return l.stream().flatMap(List::stream).collect(toList());
}
```

EXPLANATION

flatMap(List::stream) replaces each inner list with its stream and concatenates all of them into one stream.

19. Group anagrams

PROBLEM

Group a list of words into anagram groups.

EXAMPLE

```
[eat,tea,tan,ate,nat,bat]  ->  [[eat,tea,ate],[tan,nat],[bat]]
```

SOLUTION

```
static Collection<List<String>> anagrams(List<String> w) {
    return w.stream().collect(groupingBy(s -> {
        char[] c = s.toCharArray(); Arrays.sort(c); return new String(c);
    })).values();
}
```

EXPLANATION

Anagrams share the same sorted-character string, so that string is the groupingBy key. values() returns the groups.

20. Word frequency (sorted by word)

PROBLEM

Count word frequencies in a sentence, sorted alphabetically by word.

EXAMPLE

```
"the cat and the hat and the bat"  ->  {and=2, bat=1, cat=1, hat=1, the=3}
```

SOLUTION

```
static Map<String, Long> wordFreq(String s) {
    return Arrays.stream(s.toLowerCase().split("\\W+")).filter(w -> !w.isEmpty())
        .collect(groupingBy(Function.identity(), TreeMap::new, counting()));
}
```

EXPLANATION

Split on non-word characters, drop empty tokens, then groupingBy with a TreeMap factory (sorted keys) and counting().

21. Palindrome check

PROBLEM

Check whether a string is a palindrome, ignoring case and punctuation.

EXAMPLE

```
"A man, a plan, a canal: Panama"  ->  true
```

SOLUTION

```
static boolean isPalindrome(String s) {
    String t = s.toLowerCase().replaceAll("[^a-z0-9]", "");
    return IntStream.range(0, t.length() / 2)
        .allMatch(i -> t.charAt(i) == t.charAt(t.length() - 1 - i));
}
```

EXPLANATION

Normalise the string first. Then IntStream.range(0, len/2).allMatch compares each character with its mirror position, stopping at the first mismatch.

22. Primes up to n

PROBLEM

Generate all prime numbers up to n.

EXAMPLE

```
n=30  ->  [2,3,5,7,11,13,17,19,23,29]
```

SOLUTION

```
static List<Integer> primes(int n) {
    return IntStream.rangeClosed(2, n)
        .filter(x -> IntStream.rangeClosed(2, (int) Math.sqrt(x)).noneMatch(d -> x % d == 0))
        .boxed().collect(toList());
}
```

EXPLANATION

For each x in the range, noneMatch checks that no divisor from 2 to sqrt(x) divides it evenly. boxed() converts the IntStream for collecting.

23. Factorial via reduce

PROBLEM

Compute n factorial using a stream.

EXAMPLE

```
n=10  ->  3628800
```

SOLUTION

```
static long factorial(int n) {
    return LongStream.rangeClosed(1, n).reduce(1, (a, b) -> a * b);
}
```

EXPLANATION

LongStream.rangeClosed(1,n).reduce(1, (a,b)->a*b) multiplies all numbers, starting from the identity 1. It overflows a long above n=20.

24. First n Fibonacci numbers

PROBLEM

Generate the first n Fibonacci numbers.

EXAMPLE

```
n=10  ->  [0,1,1,2,3,5,8,13,21,34]
```

SOLUTION

```
static List<Long> fib(int n) {
    return Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
        .limit(n).map(f -> f[0]).collect(toList());
}
```

EXPLANATION

Stream.iterate keeps a two-element state array {a,b} and produces {b,a+b} each step. limit(n) stops the infinite stream and map takes the first element of each pair.

25. Sum of digits

PROBLEM

Compute the sum of the digits of an integer.

EXAMPLE

```
9875  ->  29
```

SOLUTION

```
static int digitSum(int n) {
    return String.valueOf(Math.abs(n)).chars().map(c -> c - '0').sum();
}
```

EXPLANATION

Convert to a string, chars() gives digit characters, and subtracting '0' maps each char to its numeric value before sum().

26. Reverse each word in a sentence

PROBLEM

Reverse each word in a sentence while keeping the word order.

EXAMPLE

```
"hello java streams"  ->  "olleh avaj smaerts"
```

SOLUTION

```
static String reverseWords(String s) {
    return Arrays.stream(s.split(" "))
        .map(w -> new StringBuilder(w).reverse().toString()).collect(joining(" "));
}
```

EXPLANATION

Split on spaces, reverse each word with StringBuilder.reverse(), and rejoin with joining(" ").

27. Longest string

PROBLEM

Find the longest string in a list.

EXAMPLE

```
[a, abcd, abc]  ->  abcd
```

SOLUTION

```
static Optional<String> longest(List<String> l) {
    return l.stream().max(Comparator.comparingInt(String::length));
}
```

EXPLANATION

max(Comparator.comparingInt(String::length)) returns an Optional of the longest element.

28. Common elements of two lists

PROBLEM

Find the elements common to two lists, without duplicates.

EXAMPLE

```
[1,2,3,4,4] and [3,4,5]  ->  [3,4]
```

SOLUTION

```
static <T> List<T> common(List<T> a, List<T> b) {
    Set<T> s = new HashSet<>(b);
    return a.stream().filter(s::contains).distinct().collect(toList());
}
```

EXPLANATION

Put the second list in a HashSet for O(1) lookups, filter the first list with set::contains, and apply distinct().

29. Merge two maps, summing values on key clash

PROBLEM

Merge two maps; when a key exists in both, add the values.

EXAMPLE

```
{a=1,b=2} + {b=3,c=4}  ->  {a=1,b=5,c=4}
```

SOLUTION

```
static Map<String, Integer> mergeSum(Map<String, Integer> a, Map<String, Integer> b) {
    return Stream.of(a, b).flatMap(m -> m.entrySet().stream())
        .collect(toMap(Map.Entry::getKey, Map.Entry::getValue, Integer::sum));
}
```

EXPLANATION

Stream.of(a,b) holds both maps, flatMap turns each into an entry stream, and toMap with Integer::sum as merge function adds values for duplicate keys.

30. Names grouped by age bucket

PROBLEM

Group employee names into age buckets (<30, 30-39, 40+).

EXAMPLE

```
EMPS  ->  {30-39=[A,C,F], 40+=[B,E], <30=[D,G]}
```

SOLUTION

```
static Map<String, List<String>> byAgeBucket(List<Emp> l) {
    return l.stream().collect(groupingBy(
        e -> e.getAge() < 30 ? "<30" : e.getAge() < 40 ? "30-39" : "40+",
        TreeMap::new, mapping(Emp::getName, toList())));
}
```

EXPLANATION

The classifier lambda maps an age to a bucket label. A TreeMap factory sorts the keys (as strings, so '<' sorts after digits) and mapping extracts names.

Part 2: Java 17 (30 problems)

Uses Java 9-17 features. The class below assumes the imports and Emp record shown.

Shared setup code

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;
import static java.util.stream.Collectors.*;

public class Streams30 {
    record Emp(String name, String dept, int salary, int age) {}

    static final List<Emp> EMPS = List.of(
        new Emp("A", "Eng", 100, 30), new Emp("B", "Eng", 150, 40),
        new Emp("C", "Eng", 120, 35), new Emp("D", "Eng", 150, 28),
        new Emp("E", "HR", 80, 45),   new Emp("F", "HR", 90, 38),
        new Emp("G", "Ops", 70, 25));
```

1. Top K frequent words (ties -> alphabetical)

PROBLEM

Given a list of words and an integer k, return the k most frequent words. Ties are broken alphabetically.

EXAMPLE

`[a,b,a,c,b,a,d], k=2  ->  [a, b]`

SOLUTION

```
static List<String> topK(List<String> w, int k) {
    return w.stream().collect(groupingBy(Function.identity(), counting()))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
            .thenComparing(Map.Entry.comparingByKey()))
        .limit(k).map(Map.Entry::getKey).toList();
}
```

EXPLANATION

Count with groupingBy + counting, sort the entries by count descending then key ascending, limit(k), and extract keys. Stream.toList() (Java 16+) returns an unmodifiable list.

2. Group anagrams

PROBLEM

Group a list of words into anagram groups.

EXAMPLE

`[eat,tea,tan,ate,nat,bat]  ->  [[eat,tea,ate],[tan,nat],[bat]]`

SOLUTION

```
static Collection<List<String>> anagrams(List<String> w) {
    return w.stream().collect(groupingBy(s -> {
        char[] c = s.toCharArray(); Arrays.sort(c); return new String(c);
    })).values();
}
```

EXPLANATION

Sorted characters form a canonical key shared by all anagrams, used as the groupingBy classifier.

3. First non-repeating character

PROBLEM

Return the first non-repeating character of a string.

EXAMPLE

```
"swiss"  ->  Optional[w]
```

SOLUTION

```
static Optional<Character> firstUnique(String s) {
    return s.chars().mapToObj(c -> (char) c)
        .collect(groupingBy(Function.identity(), LinkedHashMap::new, counting()))
        .entrySet().stream().filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey).findFirst();
}
```

EXPLANATION

A LinkedHashMap preserves encounter order while counting, so the first entry with count 1 is the answer.

4. Second highest distinct salary per department

PROBLEM

For each department, find the second highest distinct salary (empty if there is none).

EXAMPLE

```
EMPS  ->  {Eng=Optional[120], HR=Optional[80], Ops=Optional.empty}
```

SOLUTION

```
static Map<String, Optional<Integer>> secondHighest(List<Emp> l) {
    return l.stream().collect(groupingBy(Emp::dept,
        mapping(Emp::salary, collectingAndThen(toList(),
            x -> x.stream().distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst()))));
}
```

EXPLANATION

groupingBy dept, map to salaries, and collectingAndThen post-processes each group's list: distinct, sort descending, skip the top, take the first. The Optional handles departments with fewer than two distinct salaries.

5. Longest consecutive sequence, O(n)

PROBLEM

Find the length of the longest run of consecutive integers in an unsorted array, in O(n).

EXAMPLE

```
[100,4,200,1,3,2]  ->  4
```

SOLUTION

```
static int longestConsecutive(int[] a) {
    Set<Integer> s = Arrays.stream(a).boxed().collect(toSet());
    return s.stream().filter(x -> !s.contains(x - 1))
        .mapToInt(x -> (int) Stream.iterate(x, i -> i + 1).takeWhile(s::contains).count())
        .max().orElse(0);
}
```

EXPLANATION

Put values in a set. Only numbers with no predecessor (x-1 absent) start a run. For each start, Stream.iterate counts upward while takeWhile(set::contains) holds. Each element is visited a constant number of times.

6. Prefix sums

PROBLEM

Compute the running (prefix) sums of an array.

EXAMPLE

```
[1,2,3,4]  ->  [1,3,6,10]
```

SOLUTION

```
static List<Integer> prefix(int[] a) {
    return Arrays.stream(a).boxed().collect(ArrayList<Integer>::new,
        (l, x) -> l.add(l.isEmpty() ? x : l.get(l.size() - 1) + x), ArrayList::addAll);
}
```

EXPLANATION

A three-argument collect with a custom accumulator adds the previous last element to the current one. The combiner is not correct for parallel streams, so use this sequentially only.

7. Count pairs (i<j) with a[i]+a[j]==k

PROBLEM

Count pairs (i < j) with a[i] + a[j] == k.

EXAMPLE

```
[1,5,7,-1,5], k=6  ->  3
```

SOLUTION

```
static long pairsWithSum(int[] a, int k) {
    Map<Integer, Long> f = Arrays.stream(a).boxed().collect(groupingBy(x -> x, counting()));
    return f.entrySet().stream().mapToLong(e -> {
        int x = e.getKey(), y = k - x; long c = e.getValue();
        if (x < y) return c * f.getOrDefault(y, 0L);
        if (x == y) return c * (c - 1) / 2;
        return 0L;
    }).sum();
}
```

EXPLANATION

Build a frequency map. For each value x with partner y = k - x: if x < y multiply the counts, if x == y use c*(c-1)/2, otherwise contribute 0. The x < y rule avoids counting each pair twice.

8. Partition 2..n into primes / non-primes

PROBLEM

Split the numbers 2..n into primes and non-primes.

EXAMPLE

```
n=20  ->  {true=[2,3,5,7,11,13,17,19], false=[4,6,8,9,10,12,14,15,16,18,20]}
```

SOLUTION

```
static Map<Boolean, List<Integer>> primes(int n) {
    return IntStream.rangeClosed(2, n).boxed().collect(partitioningBy(
        x -> IntStream.rangeClosed(2, (int) Math.sqrt(x)).noneMatch(d -> x % d == 0)));
}
```

EXPLANATION

partitioningBy with a trial-division predicate up to sqrt(x) yields both groups in one pass.

9. Flatten arbitrarily nested lists

PROBLEM

Flatten an arbitrarily nested list of lists.

EXAMPLE

```
[1,[2,[3,4]],5]  ->  [1,2,3,4,5]
```

SOLUTION

```
static Stream<Object> flatten(Object o) {
    return o instanceof List<?> l ? l.stream().flatMap(Streams30::flatten) : Stream.of(o);
}
```

EXPLANATION

A recursive method returns a stream: for a List it flatMaps recursively, otherwise a single-element stream. The instanceof pattern variable (Java 16+) removes the explicit cast.

10. Departments by average salary, descending (ordered map)

PROBLEM

Return departments sorted by average salary descending, preserving order.

EXAMPLE

```
EMPS  ->  {Eng=130.0, HR=85.0, Ops=70.0}
```

SOLUTION

```
static Map<String, Double> deptAvgSorted(List<Emp> l) {
    return l.stream().collect(groupingBy(Emp::dept, averagingInt(Emp::salary)))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
        .collect(toMap(Map.Entry::getKey, Map.Entry::getValue, (a, b) -> a, LinkedHashMap::new));
}
```

EXPLANATION

Group with averagingInt, sort the entries by value descending, and collect into a LinkedHashMap so the order survives.

11. Highest paid employee per department

PROBLEM

Find the highest paid employee in each department.

EXAMPLE

```
EMPS  ->  {Eng=B, HR=F, Ops=G}
```

SOLUTION

```
static Map<String, Emp> topPaid(List<Emp> l) {
    return l.stream().collect(toMap(Emp::dept, Function.identity(),
        BinaryOperator.maxBy(Comparator.comparingInt(Emp::salary))));
}
```

EXPLANATION

toMap with BinaryOperator.maxBy as the merge function keeps the larger-salary employee per key. Ties keep the earlier element.

12. Min and max in one pass with teeing

PROBLEM

Find the minimum and maximum of a list in one pass.

EXAMPLE

```
[5,3,9,1]  ->  [1, 9]
```

SOLUTION

```
static int[] minMax(List<Integer> l) {
    return l.stream().collect(teeing(
        minBy(Comparator.<Integer>naturalOrder()), maxBy(Comparator.<Integer>naturalOrder()),
        (mn, mx) -> new int[]{mn.get(), mx.get()}));
}
```

EXPLANATION

Collectors.teeing (Java 12+) feeds every element to two collectors (minBy and maxBy) and combines their results with a merger function, so the data is traversed once.

13. Median

PROBLEM

Compute the median of an int array.

EXAMPLE

```
[5,1,3,2]  ->  2.5
```

SOLUTION

```
static double median(int[] a) {
    int n = a.length;
    return IntStream.of(a).sorted().skip((n - 1) / 2).limit(2 - n % 2).average().orElse(0);
}
```

EXPLANATION

Sort, skip (n-1)/2 elements, then take 1 element for odd n or 2 for even n (limit(2 - n%2)) and average them.

14. Max sum of any window of size k

PROBLEM

Find the maximum sum of any contiguous window of size k.

EXAMPLE

```
[2,1,5,1,3,2], k=3  ->  9
```

SOLUTION

```
static int maxWindow(int[] a, int k) {
    return IntStream.rangeClosed(0, a.length - k)
        .map(i -> Arrays.stream(a, i, i + k).sum()).max().getAsInt();
}
```

EXPLANATION

Generate every start index, sum the subarray with Arrays.stream(a, from, to), and take the max. This is O(n*k); a sliding window is O(n) but not natural with streams.

15. Pythagorean triples up to n

PROBLEM

List all Pythagorean triples (a <= b <= c) with c <= n.

EXAMPLE

```
n=20  ->  [3,4,5],[5,12,13],[6,8,10],[8,15,17],[9,12,15],[12,16,20]
```

SOLUTION

```
static List<int[]> triples(int n) {
    return IntStream.rangeClosed(1, n).boxed().flatMap(a ->
        IntStream.rangeClosed(a, n).boxed().flatMap(b ->
            IntStream.rangeClosed(b, n).filter(c -> a * a + b * b == c * c)
                .mapToObj(c -> new int[]{a, b, c}))).toList();
}
```

EXPLANATION

Three nested ranges via flatMap with a <= b <= c to avoid duplicates; the final filter checks a^2 + b^2 == c^2 and mapToObj builds each triple.

16. Invert Map<K,List<V>> -> Map<V,List<K>>

PROBLEM

Invert a map of key -> list of values into value -> list of keys.

EXAMPLE

```
{x=[1,2], y=[2]}  ->  {1=[x], 2=[x,y]}
```

SOLUTION

```
static <K, V> Map<V, List<K>> invert(Map<K, List<V>> m) {
    return m.entrySet().stream()
        .flatMap(e -> e.getValue().stream().map(v -> Map.entry(v, e.getKey())))
        .collect(groupingBy(Map.Entry::getKey, mapping(Map.Entry::getValue, toList())));
}
```

EXPLANATION

Flatten each entry into (value, key) pairs with flatMap, then groupingBy the value and map the pair back to the original key. Map.entry (Java 9+) creates the pair.

17. Count palindromic substrings

PROBLEM

Count all palindromic substrings of a string.

EXAMPLE

```
"aaa"  ->  6
```

SOLUTION

```
static long palSubs(String s) {
    return IntStream.range(0, s.length()).boxed()
        .flatMap(i -> IntStream.rangeClosed(i + 1, s.length()).mapToObj(j -> s.substring(i, j)))
        .filter(t -> new StringBuilder(t).reverse().toString().equals(t)).count();
}
```

EXPLANATION

Generate every substring with two nested ranges, keep those equal to their reverse, and count. O(n^3) overall, fine for interview-sized input.

18. Merge overlapping intervals

PROBLEM

Merge overlapping intervals.

EXAMPLE

```
[[1,3],[8,10],[2,6],[15,18],[17,20]]  ->  [[1,6],[8,10],[15,20]]
```

SOLUTION

```
static List<int[]> merge(int[][] iv) {
    return Arrays.stream(iv).sorted(Comparator.comparingInt(x -> x[0]))
        .collect(ArrayList<int[]>::new, (l, x) -> {
            if (l.isEmpty() || l.get(l.size() - 1)[1] < x[0]) l.add(x.clone());
            else { int[] last = l.get(l.size() - 1); last[1] = Math.max(last[1], x[1]); }
        }, ArrayList::addAll);
}
```

EXPLANATION

Sort by start. A custom collect accumulates a list: if the new interval starts after the last one ends, append a copy; otherwise extend the last interval's end with Math.max.

19. Top 3 earners per department

PROBLEM

Return the names of the top 3 earners in each department.

EXAMPLE

```
EMPS  ->  {Eng=[B,D,C], HR=[F,E], Ops=[G]}
```

SOLUTION

```
static Map<String, List<String>> top3(List<Emp> l) {
    return l.stream().collect(groupingBy(Emp::dept, collectingAndThen(toList(),
        x -> x.stream().sorted(Comparator.comparingInt(Emp::salary).reversed())
            .limit(3).map(Emp::name).toList())));
}
```

EXPLANATION

collectingAndThen turns each group's list into a sorted, limited list of names. The sort is stable, so equal salaries keep input order (B before D).

20. Sort characters by frequency desc (ties alphabetical)

PROBLEM

Sort the characters of a string by frequency descending (ties alphabetical).

EXAMPLE

```
"tree"  ->  "eert"
```

SOLUTION

```
static String freqSort(String s) {
    return s.chars().mapToObj(c -> (char) c).collect(groupingBy(c -> c, counting()))
        .entrySet().stream()
        .sorted(Map.Entry.<Character, Long>comparingByValue().reversed()
            .thenComparing(Map.Entry.comparingByKey()))
        .map(e -> String.valueOf(e.getKey()).repeat(e.getValue().intValue()))
        .collect(joining());
}
```

EXPLANATION

Count characters, sort entries by count descending then character ascending, repeat each character count times (String.repeat, Java 11+) and join.

21. First n Fibonacci numbers

PROBLEM

Generate the first n Fibonacci numbers.

EXAMPLE

```
n=10  ->  [0,1,1,2,3,5,8,13,21,34]
```

SOLUTION

```
static List<Long> fib(int n) {
    return Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
        .limit(n).map(f -> f[0]).toList();
}
```

EXPLANATION

Stream.iterate with a pair as state, limit(n), then map to the first element.

22. Duplicates in a list

PROBLEM

Find the duplicate elements in a list.

EXAMPLE

```
[1,2,2,3,3,3]  ->  {2,3}
```

SOLUTION

```
static <T> Set<T> dups(List<T> l) {
    return l.stream().collect(groupingBy(Function.identity(), counting()))
        .entrySet().stream().filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey).collect(toSet());
}
```

EXPLANATION

Count occurrences with groupingBy, keep entries with count > 1, and collect keys into a set.

23. Transpose matrix

PROBLEM

Transpose a matrix.

EXAMPLE

```
[[1,2,3],[4,5,6]]  ->  [[1,4],[2,5],[3,6]]
```

SOLUTION

```
static int[][] transpose(int[][] m) {
    return IntStream.range(0, m[0].length)
        .mapToObj(c -> Arrays.stream(m).mapToInt(r -> r[c]).toArray())
        .toArray(int[][]::new);
}
```

EXPLANATION

For each column index, build a row by taking that column from every original row. toArray(int[][]::new) assembles the result.

24. All distinct permutations of a string

PROBLEM

Generate all distinct permutations of a string.

EXAMPLE

```
"aab"  ->  [aab, aba, baa]
```

SOLUTION

```
static List<String> perms(String s) {
    if (s.length() <= 1) return List.of(s);
    return IntStream.range(0, s.length()).boxed().flatMap(i ->
        perms(s.substring(0, i) + s.substring(i + 1)).stream().map(p -> s.charAt(i) + p))
        .distinct().toList();
}
```

EXPLANATION

Recursively pick each character as the head and prepend it to the permutations of the remaining characters. distinct() removes the repeats caused by duplicate letters.

25. Count distinct pairs with difference k (k > 0)

PROBLEM

Count the distinct pairs of values that differ by k (k > 0).

EXAMPLE

```
[1,7,5,9,2,12,3], k=2  ->  4
```

SOLUTION

```
static long pairsDiffK(int[] a, int k) {
    Set<Integer> s = Arrays.stream(a).boxed().collect(toSet());
    return s.stream().filter(x -> s.contains(x + k)).count();
}
```

EXPLANATION

Put the values in a set and count each x for which x + k is also in the set. Using a set gives O(n) and automatic de-duplication.

26. Inverted index: word -> positions

PROBLEM

Build an inverted index: each word mapped to the positions where it appears.

EXAMPLE

```
[to,be,or,not,to,be]  ->  {to=[0,4], be=[1,5], or=[2], not=[3]}
```

SOLUTION

```
static Map<String, List<Integer>> index(List<String> w) {
    return IntStream.range(0, w.size()).boxed().collect(groupingBy(w::get));
}
```

EXPLANATION

Stream the indices and groupingBy the word at each index (w::get). The grouped values are the indices themselves.

27. Zip two lists

PROBLEM

Combine two lists element by element with a function (zip).

EXAMPLE

```
[1,2,3] and [a,b] with concat  ->  [1a, 2b]
```

SOLUTION

```
static <A, B, R> List<R> zip(List<A> a, List<B> b, BiFunction<A, B, R> f) {
    return IntStream.range(0, Math.min(a.size(), b.size()))
        .mapToObj(i -> f.apply(a.get(i), b.get(i))).toList();
}
```

EXPLANATION

Streams have no built-in zip, so iterate over indices up to the shorter size and apply the BiFunction to the paired elements.

28. Split list into batches of n

PROBLEM

Split a list into batches of size n.

EXAMPLE

```
[1..7], n=3  ->  [[1,2,3],[4,5,6],[7]]
```

SOLUTION

```
static <T> List<List<T>> chunk(List<T> l, int n) {
    return IntStream.range(0, (l.size() + n - 1) / n)
        .mapToObj(i -> l.subList(i * n, Math.min(l.size(), (i + 1) * n))).toList();
}
```

EXPLANATION

The number of batches is ceil(size/n). For each batch index take subList(i*n, min(size, (i+1)*n)).

29. Majority element (> n/2)

PROBLEM

Find the majority element (appears more than n/2 times), if any.

EXAMPLE

```
[2,2,1,1,2,2,3]  ->  Optional[2]
```

SOLUTION

```
static Optional<Integer> majority(int[] a) {
    return Arrays.stream(a).boxed().collect(groupingBy(x -> x, counting()))
        .entrySet().stream().filter(e -> e.getValue() > a.length / 2)
        .map(Map.Entry::getKey).findFirst();
}
```

EXPLANATION

Count frequencies and keep entries whose count exceeds length/2. At most one element can satisfy this.

30. Custom Collector: top-N largest in O(n log N) using a min-heap

PROBLEM

Write a custom Collector that returns the N largest elements in descending order.

EXAMPLE

```
[5,1,9,3,7,8], n=3  ->  [9, 8, 7]
```

SOLUTION

```
static <T extends Comparable<T>> Collector<T, PriorityQueue<T>, List<T>> topN(int n) {
    return Collector.of(
        PriorityQueue::new,
        (q, x) -> { q.add(x); if (q.size() > n) q.poll(); },
        (a, b) -> { b.forEach(x -> { a.add(x); if (a.size() > n) a.poll(); }); return a; },
        q -> q.stream().sorted(Comparator.reverseOrder()).toList());
}
```

EXPLANATION

Collector.of takes a supplier (a min-heap PriorityQueue), an accumulator (add, then poll the smallest if size > n), a combiner for parallel merges, and a finisher that sorts descending. Memory is O(n) and time O(m log n).